`allocator_traits::is_internally_relocatable`

Document #:	P3585R0
Date:	2025-01-13 11:44 EST
Project:	Programming Language C++
Audience:	LEWGI, LEWG

Reply-to:	Pablo Halpern <phalpern@halpernwightsoftware.com>

Contents

§ 1 Abstract

We propose a mechanism whereby a container implementation can bypass calls to `allocator_traits<Alloc>::construct` and `allocator_traits<Alloc>::destroy` for relocating elements internally when the allocator permits such elision. This bypass permission is already implicitly granted for allocators that do not declare their own `construct` and `destroy` members but should be a valid allowance for `pmr::polymorphic_allocator` and most other allocators. Granting this permission would allow containers such as `vector` to take advantage of optimizations provided by relocation, especially *trivial relocation* as defined in [P2786R11], even for allocators that would otherwise require a less-efficient `construct`/`destroy` sequence to effect relocation. We further propose a mechanism by which an allocator can augment the relocation operation itself, e.g., to insert bookkeeping logic before or after relocating elements. This paper builds on [P2786R11], *Trivial Relocation for C++26*.

§ 2 Change Log

R0 Pre-February 2024 Hagenberg meeting

— Initial concept introduced. Mostly-complete wording.

§ 3 Motivation

The container requirements preamble (23.2.1 [container.requirements.pre][1]) says that objects in a container must be constructed using `allocator_traits<Alloc>::construct` and destroyed using `allocator_traits<Alloc>::destroy`. Unfortunately, this requirement prevents operations such as `emplace`, `insert`, `erase`, and reallocation on `vector`-like containers from taking advantage of performance benefits from *trivial relocation*, as defined in [P2786R11], because trivial relocation bypasses constructors and destructors and therefore provides no hooks for calling the `construct` and `destroy` allocator members.

Most allocators, however, even those that provide `construct` and `destroy` members, do not require that `construct` and `destroy` be called for internal relocation of elements within a same container. We are proposing a mechanism by which allocators can inform containers that they have permission to bypass `construct` and `destroy` for such internal relocations.

Because some allocators might need to perform bookkeeping, e.g., for debugging or to track individual objects, this proposal also proposes a hook whereby the allocator can take provide a replacement relocation algorithm that would typically invoke `std::relocate`, but would add additional logic before and/or after that invocation.

§ 4 Proposed feature

We propose adding two members to `std::allocator_traits<Allocator>`:

— A function template, `allocator_traits<Alloc>::internally_relocate`, having an interface and semantic similar to the `std::relocate` function proposed in [P2786R11]. If `Alloc` declares its own `internally_relocate` member, then `allocator_traits` delegates to it; otherwise it delegates to `std::relocate`.

— A nested type trait metafunction, `allocator_traits<Alloc>::is_internally_relocatable<T>`, and its corresponding inline variable, `allocator_traits<Alloc>::is_internally_relocatable_v<T>`, that evaluate to `true_type` and `true`, respectively, if the container has permission to relocate elements of type `T` without calling `construct` and `destroy`. For a relocatable `T` (i.e, when `is_nothrow_relocatable_v<T>` is `true`), this trait's value is automatically `true` if `Alloc` has neither a `construct` nor `destroy` member function or if it has an `internally_relocate` member; otherwise, this trait's value is automatically `false`. If a specific allocator defines its own `is_internally_relocatable` metafunction,

however, then `allocator_traits` delegates to the allocator's version, just as it does for propagation traits and most other type members.

A container would be expected to invoke `allocator_traits<Allocator>::internally_relocate` instead of `std::relocate` for relocating elements within a container and would be expected to use `allocator_traits<Alloc>::is_internally_relocatable_v<T>` instead of `std::is_nothrow_relocatable_v<T>` to determine whether to use relocation for `T`.

The complexity requirements for `vector::erase` are overconstrained in a way that can be intercepted as preventing the use of relocation, regardless of the allocator. We propose a small wording change for that, too.

§ 5 Before/After Comparisons

Below is a snippit from an implementation of `vector<T, A>::erase`, where `first` and `last` define the range of addresses for the elements being erased. The *Before* code restricts relocation to cases where the allocator has neither a `construct` nor `destroy` function. The *After* code expands the use of relocation to include allocators that give explicit permission to do so. Although the *After* code is shorter, the goal is not concision, but the functionality of extending relocation to a broader set of allocators.

Before

```
using AT = allocator_traits<A>;
if constexpr
        (is_nothrow_relocatable_v<T>
        &&
              ! requires(A& a, T* p)
        {
                  a.construct(p,
        std::move(*p)); } &&
              ! requires(A &a, T* p)
        { a.destroy(p); }) {
  AT::destroy(m_alloc, first);
  m_end = relocate(last, m_end,
        first);
}
```

After

```
using AT = allocator_traits<A>;
if constexpr (AT::template
        is_internally_relocatable_v<T>)
        {
  AT::destroy(m_alloc, first);
  m_end =
        AT::internally_relocate(m_alloc,
                                last,
        m_end, first);
}
```

§ 6 Alternatives considered

The proposal in this paper provides maximum flexibility at the cost of some complexity. There are two orthogonal ways we can consider for reducing flexibility to gain simplicity and concision.

§ 6.1 Replace `is_internally_relocatable` metafunction with a fixed value

`allocator_traits<A>::is_internally_relocatable<T>` is a metafunction whose value is dependent both on the allocator, `A`, and the element type, `T`. The evaluation of this metafunction is made especially verbose by the necessary use of the `template` keyword:

```
if constexpr (allocator_traits<A>::template is_internally_relocatable<T>)
  { /* use relocation */ }
```

When overriding this trait for a specific allocator we have yet to see an example where this trait is other than an alias for `std::is_nothrow_relocatable`; i.e., we have never seen an allocator that needs to make a per-type decision on relocatability that is different from the default.

One simplification, then, would be to replace the `is_internally_relocatable` metafunction with a simple `true_type`/`false_type` metavalue that would be independent of `T`, and thus easier to override for a specific allocator. This metavalue would be `true_type` if the allocator is granting permission to perform relocation without invoking `construct` and `destroy`, but would not convey any information about whether `T` itself is relocatable:

```
if constexpr (std::allocator_traits<A>::allow_internal_relocation &&
              std::is_nothrow_relocatable<T>)
  { /* use relocation */ }
```

The `allow_internal_relocation` metavalue is syntactically cleaner and easier to override, but is not really simpler to use because `is_nothrow_relocatable` must now be evaluated separately, so this alternative was considered slightly inferior to the current proposal.

§ 6.2 Do not provide the `internally_relocate` member

It is probably rare that a specific allocator type would mandate that internal relocation be performed using anything other than a call to `std::relocate`. This proposal could be simplified, therefore, by removing the `internally_relocate` function template.

We have found in testing, however, that being able to instrument calls to `internally_relocate` is useful for verifying that relocation is occurring when expected. Furthermore, allocators have been described that keep track of individual elements, and it seems unwise to close off the benefits of relocation for such allocators. So, while, `internally_relocate` slightly increases the specification size and implementation burden, we concluded that the increased flexibility warrants its inclusion.

§ 7 Implementation Experience

A working implementation of the modified `allocator_traits` and a `vector` that takes advantage of the new feature can be found at https://github.com/MungoG/relocatability/tree/master/code.

There is extensive experience bypassing `construct` and `destroy` for relocation when using a pmr-like allocators in the BDE library.

§ 8 Wording

The wording below is lacking changes to header synopsis and class definitions, but should provide sufficient detail for evaluating this proposal.

To **Allocator Traits Member Types**, 20.2.9.2 [allocator.traits.types], add

```
template <class T> struct is_internally_relocatable;
```

> *Class Template*: `is_internally_relocatable<T>::value` is a `constexpr bool` whose value is:
>
> — `Alloc::is_internally_relocatable<T>::value`, if such a value exists; otherwise
>
> — `is_nothrow_relocatable_v<T>` if `Alloc::internally_relocate(T*, T*, T*)` is an invocable member; otherwise
>
> — `false` if either or both of `Alloc::construct` or `Alloc::destroy` are invocable members; otherwise
>
> — `std::is_nothrow_relocatable_v<T>`.
>
> `constexpr bool is_internally_relocatable_v<T>` is an alias for `is_internally_relocatable<T>::value`.

To **Allocator Traits Static member functions**, 20.2.9.3 [allocator.traits.members], add

```
template <class T>
  T* internally_relocate(Alloc& a, T* first, T* last, T* result);
```

> *Mandates*: `is_internally_relocatable_v<T>` is `true`.
>
> *Preconditions*: `(last - first) != 1`, or `*first` points to a complete object (6.7.2 [intro.object]).
>
> *Effects*: Calls `a.internally_relocate(first, last, result)` if such an expression is valid, otherwise, calls `std::relocate(first, last, result)`.
>
> *Returns*: `result + (last-first)`.
>
> *Throws*: `nothing`

In **Class `polymorphic_allocator` General**, 20.4.3.1 [mem.poly.allocator.class.general], add a `is_internally_relocatable` member metafunction:

```
public:
  using value_type = Tp;
  template <class U>
    using is_internally_relocatable = is_nothrow_relocatable<U>;
```

Change **Vector modifiers** 23.3.11.5 [vector.modifiers] as follows:

```
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();
```

Effects: Invalidates iterators and references at or after the point of the erase.

Throws: Nothing unless an exception is thrown by the assignment operator or move assignment operator of T.

Complexity: ~~The destructor of T is called the number of times equal to the number of the elements erased, but the assignment operator of T is called the number of times equal to the number of elements in the vector after the erased elements.~~ Linear in the distance from the first element to be erased to the end of the original vector.

§ 9 References

[P2786R11] Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. 2024-12-17. Trivial Relocatability For C++26.
https://wg21.link/p2786r11

---

1. All citations to the Standard are to working draft N4993 unless otherwise specified.↩